

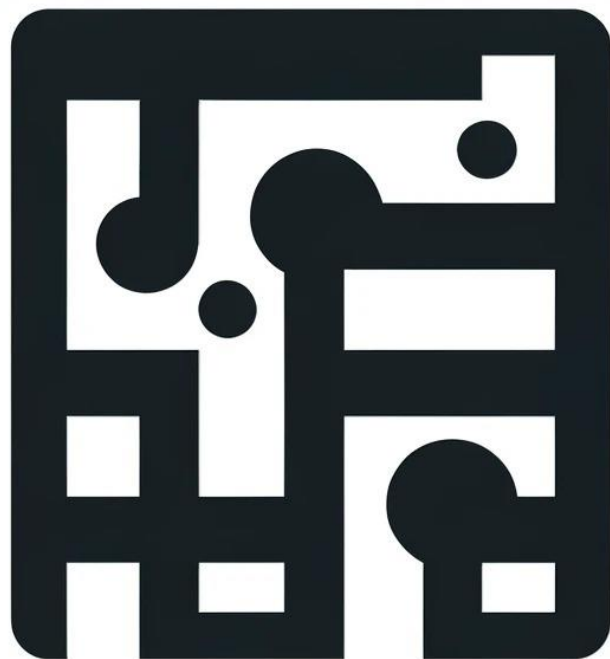
Project Platform

INF – 2900 Software Development

Team 15

August Karlsen, Markus Wilhelmsen, Mahmoud Allorqsosi and Tor Opdahl

January 31st – May 16th



Project Platform

Innhold

Innhold	1
Introduction.....	4
Technical Background	4

Scrum.....	4
Django	4
React.....	5
Languages.....	5
Project Content	5
User Stories	5
Scenarios	6
Personas	6
Design	7
System Architecture	7
Front-End Design	7
Back-End Design	8
Database Design	9
User and Profile	9
Posts and Tags	10
Comments and Reactions	10
Followers and Notifications.....	10
ORM and Model Management	10
Migration Management	10
Data Validation and Integrity	11
Performance and Optimization	11
Indexing	11
Query Optimization	11
Security.....	11
Access Control	11
Data Protection	11
Implementation.....	11
Frontend.....	11
Navigation and Uniform Resource Identifiers.....	11
Authorization and authentication.....	12
Application components	12
Backend	14
URL Management.....	14
User Profile Management	15
Models.....	15
Serializers	16

Views	17
AdvancedPostSearchView	17
Process Content	17
Development Cycle	17
Test Driven Development	18
Discussion and Reflection	19
Product Development	19
Test Driven Development Reflection	19
Group/Project work Reflection	20
Challenges	20
Conclusion	20
References.....	20
Appendix	21
User Stories List	21
MoSCow Analysis	23
Backlog	24
Timesheet and Contributions.....	24
Meeting Minutes	25

Introduction

Product Ideas and Vision

The vision for this project was to create a social media platform that allows users to post content and interact with others' content. The objective was straightforward: to develop a platform where users can easily share their thoughts, experiences, and engage with a community.

Product Goals and Description

The primary goal of this social media application is to provide users with a seamless and intuitive experience. Users should be able to create posts, comment on others' posts, and follow other users to stay updated with their content. The platform aims to foster meaningful interactions and build a community where users can connect and share their interests.

Group Introduction

Team 15 consists of August Karlsen, Markus Wilhelmsen, Mahmoud Allorqsoosi, and Tor Opdahl. Each team member contributed to different aspects of the project, including backend development, frontend development, and overall design. Our collaborative efforts were focused on creating a functional and user-friendly social media platform.

Technical Background

Scrum

In software development, the Scrum framework has become well-known for providing an organized technique for project management and teamwork. Scrum, which is based on agile principles and represents flexibility, openness, and iterative development, is especially well-suited to large and dynamic computer science projects. Scrum establishes the roles, products, and activities that move the development process forward. Scrum clearly defines roles and encourages teamwork, with the Product Owner, Scrum Master, and Development Team being examples of such roles.

Teams may prioritize work and give value gradually by using project requirements and progress visibility provided by artifacts like the Product Backlog and Sprint Backlog. In the meantime, continual communication, feedback, and reflection are ensured by occasions like sprint planning, daily standups, sprint reviews, and sprint retrospectives, which encourage ongoing adaptation and progress. Computer science projects can gain from improved collaboration, adaptability, and reactivity by adopting Scrum's tenets, which will ultimately result in the effective delivery of high-quality software solutions.

Django

In web development, the Django framework is an invaluable resource, especially when it comes to rapidly and effectively creating online applications. It has a straightforward, well-organized structure that divides the various sections of the code and is based on Python.

Many pre-built capabilities in Django make basic tasks, such as working with databases and developing user interfaces, much simpler. For instance, it features an ORM-based data handling

framework that streamlines database interactions. It also includes tools for creating dynamic web pages and managing URLs.

One of the most beneficial things about Django is that it promotes readable and well-written code among developers. It accomplishes this by giving developers a clear project structure and logical defaults, which frees up their time to write quality code rather than tedious activities.

React

In web development, React is a common framework for creating user interfaces. By dissecting the interface into reusable components, it makes the process of developing dynamic and interactive online apps easier. Complex application management and upkeep are facilitated by these components, which capture the behavior and structure of UI elements.

React's virtual Document Object Model (DOM), which effectively refreshes the user interface by only rendering the modified components, is one of its primary characteristics. As a result, compared to conventional methods, performance is enhanced, and the user experience is optimized.

Additionally, React encourages declarative programming, in which programmers specify the desired interface design based on the state of the application at that moment. This allows developers to concentrate on the intended result rather than the detailed steps involved in reaching it, which facilitates reasoning through and debugging code.

React also boasts a robust ecosystem of tools and frameworks, including Redux for state management and React Router for navigation management, that expand its functionality and make it simple for developers to create complex apps.

Languages

The project utilizes Python for back-end development, while HTML, CSS, and JavaScript are employed for front-end development. These languages collectively form the foundation of the project, facilitating both server-side and client-side functionality.

Project Content

User Stories

User stories have been essential to ProjectPlatform's development process in helping us make decisions. These tales show the range of consumer demands and the technological specifications necessary for a strong platform.

Features for permission and authentication are simple to use, making user onboarding effortless. Users may easily create safe accounts, log in, and log out. To ensure seamless operations, administrators can also control user roles and permissions.

Users may easily create, browse, edit, and remove content from blog entries, and there are rich text formatting options available for interesting postings. Deeper connections and interactions within the community are fostered by the commenting system, which allows users to voice their opinions and participate in threaded discussions.

By enabling users to follow other people and customize their activity stream according to their interests, user engagement elements further improve the overall experience. Users are empowered to efficiently organize postings with tagging and categorization features, which improve discoverability.

Users are given more flexibility and usefulness by optional extra features including reaction buttons, notifications, powerful search, draft saving, and multi-language support.

Our system architectural decisions, which combine Bootstrap or Tailwind CSS for styling with React, Vue.js, or Angular for dynamic interfaces, guarantee a fluid and aesthetically pleasing user experience. The foundation is Django, which takes media processing and data storage into account.

The setup and installation instructions are brief and easy to understand; they cover local deployment, database configuration, dependency installation, and Python environments. API documentation makes it easier to integrate with other systems by providing thorough insights into endpoints and request/response formats.

Wireframes and design system standards in UI design guarantee uniformity and usefulness throughout the platform. Version control, code review procedures, and efficient communication routes are all included in collaboration rules, which promote a productive and cooperative working environment.

See [Appendix A](#) for complete list of User Stories.

Scenarios

John is an enthusiastic blogger who wants to use the Blog Platform to tell his fans about his most recent trip. After logging in, he goes to the dashboard and selects the option to start a new blog post. He eagerly selects the "Create New Post" option, opening a form where he can enter the post's title, content, and pertinent tags. He enriches his content with evocative descriptions and compelling photographs of his journey by using the powerful text editor that is offered.

John hits "Publish" after he is happy with his post, which instantly makes his trip story accessible to his followers. He's waiting impatiently for their responses when he gets a notification that there's a new comment on his post. He is curious, so he clicks on the notification to discover that one of his fans has left a sincere comment praising his spirit of adventure. John responds to the message, delighted to have sparked an active discussion among his followers.

Later, as John continues to go deeper into the Blog Platform, he finds the "Following" option, which enables him to receive updates on posts made by users he finds inspiring. Eager to experience new adventures and meet like-minded people, he chooses to follow a few other travelers whose tales motivate him.

When John thinks back on his time using the Blog Platform, he values its extensive features and easy-to-use interface, which encourage meaningful relationships among bloggers. He is excited to share his experiences and establish connections with other enthusiasts worldwide as he continues his blogging journey on the site.

Personas

John is a free-lance writer in his early thirties who uses his blog to express his ideas and experiences. Because he values his time and manages several writing projects at once, he gives priority to technologies that are user-friendly and effective. John looks for solutions with simple workflows and intuitive user interfaces so he can concentrate on creating engaging content without interruptions.

John hates slow or glitchy software that limits his creativity and productivity. Instead, he demands fluid and responsive user experience. To clearly articulate his thoughts and engage his audience, he

depends on trustworthy sources. John is drawn to platforms that put the user experience first and provide tools specifically designed to meet his needs as a blogger.

Rich text editing, labeling, and classification are some of the features he appreciates that help with content management and accessibility. Furthermore, John values interactive features like following and commenting since they encourage community participation and feedback on his work.

John likes to conduct study on new subjects and work as a team with other writers to share ideas and ideas during his free time. Apart from meeting his work-related responsibilities, he is looking for a blogging platform that facilitates personal development and networking. As he starts his blogging career, John wants a platform that will enable him to express himself artistically and make his voice heard online.

Design

System Architecture

The architecture of the platform is divided into three main layers:

1. The Client-Side(front-end)
2. The Server-Side(back-end)
3. The Database

The front-end is built using React. We chose to use React as our front-end because of the many good features and effectiveness of the framework. Some of the features include; component-based architecture, virtual DOM, declarative syntax, JSX, react hooks and performance optimizations.

The front-end communicates with the back end using Axios to send HTTP requests. Axios also comes with good features, as it allows for easy creation of HTTP requests in JS, Axios is also promise based, meaning asynchronous code is more readable and manageable. Axios also provides request and response transformation, this is helpful as responses can easily be serialized as the data is JSON formatted, and the API endpoint in the backend is easily reached.

The back end is implemented using Django, more specifically Django's RESTful API. Django provides a robust framework for development and deployment of a scalable API that meets the requirements of the platform. Django offers many different features when it comes to back-end development, such as the user authentication mechanisms, URL routing, serialization, view classes and query handling.

The API endpoints and handling HTTP requests utilizes the user authentication mechanisms, view classes, URL routing and serialization capabilities Django offers. These features of the framework have streamlined the development of the backend when it comes to communication with the frontend. The user-friendly interface of the RESTful API Django provides is very helpful for development as it allows for easy testing and interaction with the different API endpoints.

The database layer consists of a SQLite3 database, which forms the backbone for the application. SQLite 3 offers robust features such as ACID compliance, transaction support and full SQL functionality.

Front-End Design

User Interface

The user interface is designed to be easy to use for the users, and the platform is made to be easy to navigate. Simplicity and minimalism are one of the main focuses of the platform's design. By focusing

on these aspects, it reduces the cognitive load and helps the user use the platform without any instructions, as every feature is intuitively implemented and presented. When entering the platform, the user lands on the landing page, which displays the newest posts made by all users, with a navigation bar at the top for the convenience of the user. The simple design is there so that no user should get frustrated by using the platform. When it comes to a user posting their own posts, a simple box is at the bottom of the screen, making it easy for a user to express or share what they want.

Ascetics/Color Palette

The colors the platform uses are a selected combination of soothing blues and dark accents. This selected combination was chosen mainly to be a pleasing experience for the eye of the user, while also keeping in mind users tend to engage more on sites with such a theme.

Responsive Design

The front-end uses the capabilities of the React framework, making the front-end highly responsive as only the necessary components need to be re-rendered when data changes. This, in addition to being good for server load management, also makes it a better user experience as there is no need for a full page reload for data to be updated. For example, when a user reacts to a post, only the reaction component needs to be updated, not the whole page.

Back-End Design

API

The back end of the platform is built using Django and its REST framework. This framework facilitates rapid development and deployment of APIs and database management for web applications. The REST framework is a powerful toolkit, capable of handling various types of HTTP methods such as GET, POST, PUT, and DELETE.

URL Routing and Endpoints

When the Django backend receives a request from the frontend, it uses the URL to identify the API endpoint using Django's URL routing mechanisms. Well-defined URL configurations in `api/urls.py` map the incoming requests to specific views in `api/views.py`. This ensures that each request is directed to the appropriate handler, enabling the backend to respond correctly.

Views

The views in `api/views.py` are responsible for handling the requests and performing the necessary operations on the data. For instance, the `AdvancedPostSearchView` leverages.

Diagram: API Request Flow.

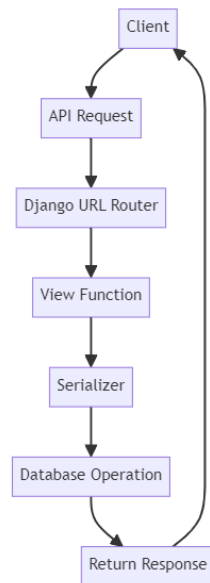


Figure 1: This diagram illustrates the flow of an API request in the backend. The client sends an API request, which is routed by Django to the appropriate view function. The view function uses serializers to interact with the database, processes the request, and returns a response back to the client.

Serializers

In `serializers.py`, serializers define the structure of the data returned by the API. The `PostSerializer` is an example of a serializer used to convert `Post` model instances to JSON format, ensuring that data is properly structured when sent to the frontend.

RESTful Principles

The back-end framework adheres to RESTful principles, ensuring that interactions between the front-end and back-end are stateless and that there is a clear separation between client and server. Each API endpoint corresponds to a specific resource or set of resources managed by Django's Object-Relational Mapping (ORM) system. The ORM enables efficient database operations using Python code instead of raw SQL queries, simplifying data manipulation and enhancing productivity.

Security

The platform utilizes Django's robust authentication and authorization mechanisms. For instance, user authentication is managed via JWT tokens, ensuring secure access to protected endpoints.

By adhering to these principles and leveraging the capabilities of Django and its REST framework, the platform ensures secure, efficient, and scalable API interactions.

Database Design

The database is designed to handle various entities such as users, posts, comments, profiles, tags, categories, followers, reactions, and notifications. Below is an overview of the key models and their relationships:

User and Profile

- Each user has a one-to-one relationship with a profile, storing additional information such as photo, biography, and location.

Posts and Tags

- Posts are linked to users and can have multiple tags and a single category.
- Tags are associated with multiple posts through a many-to-many relationship.

Comments and Reactions

- Comments are linked to posts and users, with the ability to form threaded discussions.
- Reactions (like/dislike) are linked to posts and users.

Followers and Notifications

- Users can follow each other, forming many-to-many relationships through a Followers model.
- Notifications are linked to users and provide alerts for various interactions.

Diagram representing the database:

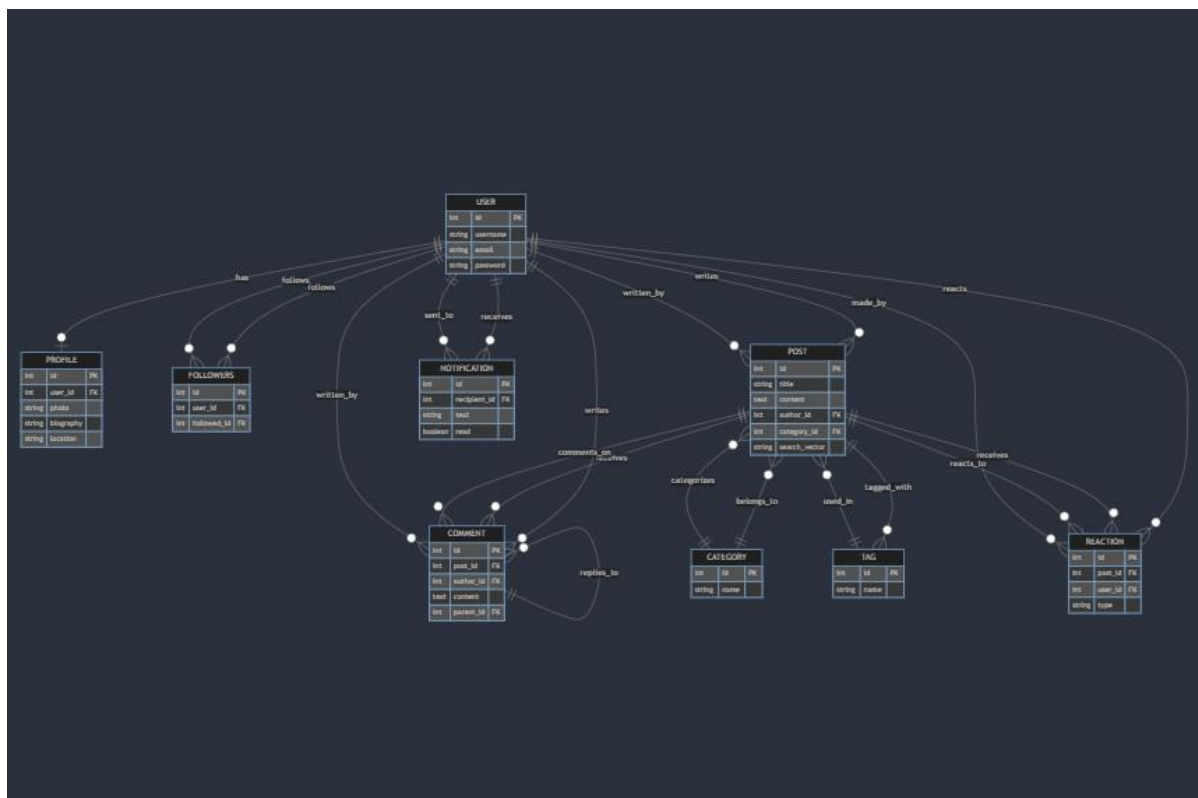


Figure 2: This diagram represents the database schema for the social media platform. It includes tables for users, profiles, followers, posts, tags, categories, comments, reactions, and notifications, along with their relationships.

ORM and Model Management

Django's ORM (Object-Relational Mapping) system simplifies database interactions by allowing us to use Python code to interact with the database. Each model represents a table in the database, and instances of models correspond to rows in those tables.

Migration Management

Migrations are used to propagate changes to the database schema. Each change in the model definition creates a new migration file that can be applied to update the database schema.

We use Django's migration system to handle schema changes, ensuring that the database remains synchronized with the models.

Data Validation and Integrity

Django's model fields provide built-in validation, ensuring that only valid data is stored in the database.

Relationships between models (foreign keys, many-to-many, one-to-one) are enforced at the database level, ensuring referential integrity.

Performance and Optimization

Indexing

We use database indexing to speed up queries, particularly for fields frequently used in search operations. For example, the `search_vector` field in the `Post` model uses a GIN index to optimize full-text search queries.

Query Optimization

Django's ORM allows for query optimization techniques such as `select_related` and `prefetch_related` to reduce the number of database queries and improve performance.

Security

Access Control

User authentication and authorization are managed through Django's built-in user model and authentication system. JWT tokens are used for secure access to protected endpoints.

Data Protection

Sensitive data is encrypted, and access controls are in place to ensure that only authorized users can access certain information.

By leveraging Django's robust ORM and database management capabilities, we ensure that our platform's data is handled efficiently, securely, and reliably.

Implementation

Frontend

The frontend of the web application was built using React.js, a JavaScript framework which allows plenty functionality and helps bring the application to life.

Navigation and Uniform Resource Identifiers

With React we can utilize routing between publicly and privately visible sites and components. A wrapper is simply put a component which when encapsulating other components, provides them

with the variables and functionality of the wrapper component. In our case, we define a router wrapper to encapsulate other components such as an authorization provider and a profile provider, which also serves as wrappers for the components encapsulated within. These two wrappers authorize the user and return the given profile for that user. The routing and authorization wrappers play a significant, if not the most important, role in the application; authentication and authorization. At the core, each user-accessible path of the application is a component which defines the specific path, and the page to which the path leads. This page component is wrapped inside a private route element which defines that this is only accessible to authorized users, meaning any user that is logged in.

Authorization and authentication

Within these wrappers we define what an authorized user is able to see and what an unauthorized user isn't. Components such as your private profile and the details of a post, are components only an authorized user has access to. Meanwhile, the login and error page are components any user could access.

A function within the authorization wrapper component sends a request to the backend, which in return gives the user a set of JSON Web Tokens, or JWT for short. These tokens are used to authenticate the user and ensure that the user is authorized to access certain pages and components of the application. The authorization component contains functions to log in a user and sign up a new user. The return value for the login is the JWT token, which is passed as context for the components encapsulated in the authorization wrapper. The wrapper also exports functions such as log in, log out and sign up, which is mainly utilized by the login page.

The majority of the requests sent to the Django server from the React application includes the authorization header in the request. This authorization header contains the access part of the JWT token. This is because the Django server uses this token to authenticate the user and retrieve data based on the user.

As you've read, there is a cooperation between Django and React on this front. The frontend asks for a token to authorize the user and allow access to its components, and the backend returns such a token to the frontend, which itself uses to authenticate the user. As long as the user retains this token, the user is allowed to access the web application, and the server knows who's accessing.

There also exists a profile component wrapper, which relies on the user already being logged in. This component fetches the profile from the database, and exports both the profile data and the function to retrieve profile data to its encapsulated children. The reason for the function, is that whenever a user updates their profile, it must be fetched again to display the updated data.

Application components

The application utilizes several functions provided by React. These functions contribute to the overall UX/UI aspect of the application.

- The first to mention is a function which utilizes the wrapper, such as the ones for authorization and profile. It retrieves the variables and functions of the wrapper component.
- Second, we have a function to assign and access data, this promotes structure and organization to the code.

- At last is a function which enables execution of specific code based on a set of variables called the dependencies. For example, for a public profile component which uses the username part of the URL within its dependency array, a user might trigger this function to run when accessing another username. Which essentially is the same page, but with different data, as the function has been triggered by the dependency changing.

Header component

The header is less a page more a component. It is rendered on all pages of the application and serves as navigation to the user. Each element leads to a specific page, such as the profile or search.

Login page

The login page is a simplistic component which displays a set of input fields with a logo. If the user wishes to sign up instead, the form expands and the function called when submitting is also changed. These submission functions are provided by the authorization provider. There also exists a timeout on the submission, to assure no unauthorized user could spam the server with requests.

Landing page

Once the user is logged in, the landing page is the first sight to see. This is where the user can view and interact with posts uploaded by other users. The landing page consists of the functions to request posts, categories and tags, from the database. In the scenario that a user would want to share something with other users, there are several functions in place to handle the input of text, category and tag change. Once the user is ready to submit the post, a POST request is sent to the server to store the post in the database.

The landing page itself renders the form to create a post. The rendering of posts is managed by another component called inside the landing page. This component inputs post data and displays the post and its additional details, the input data is an array of posts, where the component maps each of these using the id. The additional details of such a post is the name and username of the person who posted it, as well as the photo for this user. This component also defines a delete function for the post, which sends a delete request to the server.

Post and comment details

The post details component renders a single view of a post, as well as any comments posted by other users. In addition to this, there is also a form to which a user can comment on this post. The component retrieves specific post details and additional comments by request to the server.

Almost identical to the post component, is the comment details component. This component renders specific comment details and allows a user to comment on the comment. Essentially creating a sub comment.

Profile and public profile page

The profile page is a component which renders the data from the profile of the user. The component defines several functions used to enhance the user experience, as well as to provide the component with necessary variables.

There are two functions which acts as a toggle when pressed by a button which is rendered on the profile. The button pressed for changing a profile photo, reveals a photo submission form. The button pressed for changing the profile, reveals a form to submit changes to the profile, such as the biography. Both of these sends a POST request to the server when submitted, applying the changes to the database.

In addition to rendering details of the profile table of the user. The component also renders the specific user's posts, with the help of the same component used in the landing page. As previously mentioned, this component inputs the data retrieved from the server, and in return displays information about the posts based on that data.

There is an additional function which retrieves the users posts by request to the server in the main component.

Search

The search component is a small but elegant. It defines a GET request to the server with the user search as input, the server will then return the data found which matches the search by either username or post content. A second function is defined to handle the input of the search form, if the search value exceeds two in length, the request function is called to initiate the search and display the results immediately. If the search value drops below two in length, all search results are removed. This helps the user feel like the search is happening instantaneously, as it is, but without the need for pressing a button.

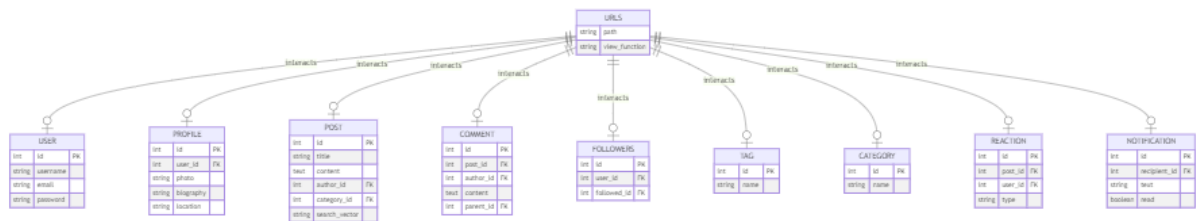
The result from the search is rendered in another component. This component inputs a JSON which contains an array of users and an array of posts. Each array is displayed using mappings by id. The array of users displays the name and username of the user, and the array of posts is essentially the same as the posts component, but the content of the post is limited by a specific number of characters.

Reactions

The reaction component is responsible for fetching the user's reaction together with the total reactions count and comment count on a post. This component sends request to the backend when the user wants to; react to a post, change the current reaction and delete the reaction they have on a post. Each of the requests from the backend sends an updated response on what reactions are on the reacted to post by the user, so the frontend updates this on receiving, making reactions highly responsive.

Backend

URL Management



The `urls.py` file is essential for defining the URL patterns that map to view functions. It organizes and directs incoming HTTP requests to the corresponding views, ensuring the correct handling of actions within the Blog Platform.

User Profile Management

The management of user profiles, authentication, posts, comments, sub-comments, tags, and categories are all taken care of by the API endpoints. Paths connected to user profiles include obtaining and updating profile data, obtaining and uploading profile photos, and looking up user profiles that are visible to the public. Token management and user registration are made easier by authentication and authorization endpoints. Post management pathways include actions like getting all posts in the database and creating, removing, and retrieving posts. The generation and retrieval of comments as well as their responses are made possible via the comment and sub comment management APIs. Lists of all categories and tags are provided by tag and category management endpoints. Lastly, while media files are being developed, static files are served.

Models

The foundation of our Blog Platform is built upon several key Django models that structure the data and relationships within the application. These models facilitate the organization, categorization, and interaction with the content on our platform. Below is a detailed description of each model and its role in the application:

Base Model

The Base Model class is an abstract model that provides common fields for all models in the application. This ensures consistency and reduces redundancy across all models by including creation

and update timestamps. The explanations below will not go through the fields of each individual model, or its relationships as they are described in the design part.

Tag

The Tag model is used for categorizing posts with specific keywords. Each tag is uniquely named and helps in filtering posts based on common themes or subjects, enhancing the navigability and user experience of the blog.

Category

The Category model organizes posts into broader topics. Like tags, categories facilitate grouping posts into general themes, making it easier for users to find content within specific domains of interest.

Post

The Post model represents the core content of the blog platform. It includes essential attributes such as the title, content, author, and relationships to tags and categories. This model is central to the platform's functionality, enabling the creation, display, and management of blog content.

Comment

The Comment model supports user engagement through comments on posts. It includes fields for the comment's content, its author, and an optional parent comment to enable threaded discussions. This model enriches the blog by allowing readers to participate in discussions, share feedback, and interact with both the content and other readers.

Profile

The Profile model extends the user profile with additional attributes such as a photo, biography, and location. It provides a more personalized user experience.

Followers

The Followers model tracks the relationships between users, enabling the following functionality.

Reaction

The Reaction model captures user reactions (like/dislike) on blog posts.

Notification

The Notification model handles user notifications triggered by various interactions such as new reactions or comments on their posts.

Serializers

Serializers in Django REST framework is used to convert complex data types such as Django models into JSON format and vice versa. They ensure that the data exchanged between the server and the client is validated, serialized, and deserialized correctly. Our project utilizes several serializers to handle various data transformations for users, profiles, posts, comments, reactions, and notifications.

All the different serializers is implemented similarly, as they do the same job, but for different models in the database. There will therefore not be explained what each serializer does, but rather a general description of how they are implemented.

Each serializer class is set up with a Meta class that specifies the model and fields to be serialized. The serializers validate the data before it is saved to the database, ensuring data integrity and

consistency across the application. The serializers are also used for translating the model instances in the implementation into JSON formatted data for when data is to be sent to the frontend.

Views

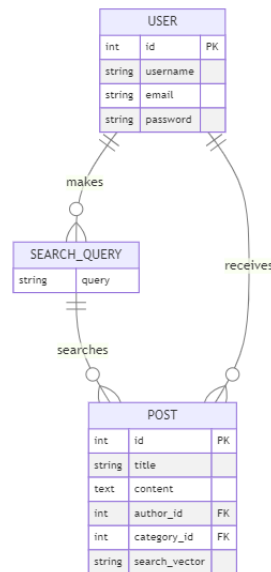


Figure 3: This diagram illustrates the flow of an API request in the backend. The client sends an API request, which is routed by Django to the appropriate view function. The view function processes the request, interacts with the database using serializers, and returns a response back to the client.

The views.py file in our project contains the logic for handling HTTP requests and responses. This section outlines the key views implemented in the Blog Platform, focusing on the advanced search functionality provided by the AdvancedPostSearchView.

AdvancedPostSearchView

The AdvancedPostSearchView is a class-based view that extends Django REST framework's ListAPIView. It is designed to provide a robust search feature for blog posts, leveraging PostgreSQL's full-text search capabilities. This view allows users to search for posts by their titles and content, with results ranked by relevance.

Purpose and Benefits

The AdvancedPostSearchView enhances the user experience by providing an efficient and effective search mechanism. Users can quickly find relevant posts, improving the overall usability of the platform. The use of weighted search vectors ensures that more relevant results (e.g., matches in the post title) are prioritized, delivering a more intuitive and satisfying search experience.

Process Content

Development Cycle

The Agile development cycle has been followed by our project, although there have been a few departures from its principles. Even though we haven't stuck to the Agile method exactly, we have included crucial components to direct our development process, like keeping a backlog, performing MOSCOW analyses, and creating user stories. These materials have given project requirements and task prioritization an organized framework.

One significant exception from the strict Agile technique is the team members' individual work selection. During sprint planning, instead of assigning tasks collectively, each developer has been given the freedom to choose tasks according to their own interests or areas of expertise. Team members' sense of accountability and ownership has been increased by this strategy, but there have also occasionally been differences in job distribution and project priority alignment.

Two-week sprints are the standard rhythm for our project, giving development work a consistent pace. The team's goal has been to offer a set of planned features or enhancements at the end of each sprint, iterating feedback and gradually improving functionality. Sprint format has allowed for flexibility in responding to unforeseen obstacles or changing requirements while maintaining a steady rate of progress.

The assistance teacher's (TA) function as the scrum master in our Agile setup has been crucial for encouraging the Agile approach. In addition to facilitating sprint planning meetings and ensuring that the team follows Agile principles, the TA has given advice on best practices for Agile. Weekly check-ins with the TA have also functioned to assess progress, resolve roadblocks, and modify plans as needed.

We're an online group, but we've managed to collaborate and communicate regularly by getting in touch on Discord once a week. Team members have been able to coordinate activities, exchange updates, and aid when needed thanks to this continuous communication.

Test Driven Development

In developing the platform, we have followed the test-driven developments (TDD) principles to some extent. This means that the principles were not always followed during implementation. As the basic flow of the TDD is to make the test first, and then write the code that should pass the created test failed on some occasions due to eagerness of finishing the different features and creation of bad tests that needed changing as the logic behind them were not sound.

There are both good and bad examples of following the TDD's principles during the implementation of the platform. Just to name one, is when implementing the view for when a user wants to create a new post and store it in the database. The test then sees if the view acts according to its implementation and does what it needs to do.

However, the times where the TDD's principles were not followed is for example in the process of making the views for the creation of new reactions. The test was formatted badly, such that the reactions view did not have the information it needed to have for it to work. This flaw was firstly discovered during the implementation of said view, rendering the test useless as it had no hope of passing it. The changes were so drastic during the development that the test was first finalized after the development of the view was finished.

All group members were new to this concept in the beginning of the development. It was rough adjusting to the new way of implementing functions, however we all saw the value in utilizing this strategy, as we all saw the benefits of the approach. When we utilized the strategy correctly, we saw how visualizing and planning on what the function should do with said parameters made the

development of the function itself easier and the logic became more obvious. This also made the functions simpler and had less unnecessary operations. In short, following the principle made better code in comparison to when failing to follow the principle.

Discussion and Reflection

Product Development

Working on the backend for our project has been an enlightening experience, providing several key takeaways:

- Understanding of Django Framework:

I gained a deeper understanding of the Django framework, especially its ORM (Object-Relational Mapping) system, which simplifies database interactions.

The Django Rest Framework (DRF) proved to be an invaluable tool for quickly developing RESTful APIs, enhancing my skills in building scalable and efficient web services.

- API Design and Implementation:

Designing and implementing APIs required careful planning to ensure that the endpoints were both functional and secure. I learned how to create well-structured and documented APIs that could be easily consumed by the frontend.

I also learned about best practices for API versioning, rate limiting, and error handling to ensure a robust and user-friendly backend.

- Authentication and Authorization:

Implementing JWT (JSON Web Tokens) for authentication provided a practical understanding of secure user authentication and authorization mechanisms.

Managing user sessions, securing endpoints, and ensuring that only authorized users could access certain resources were critical components of the development process.

- Database Management:

Working with Django's ORM, I learned how to define and manage complex relationships between different models, such as users, posts, comments, and reactions.

I also gained experience in optimizing database queries and ensuring data integrity through proper use of transactions and validations.

- Collaboration and Version Control:

Using Git and GitHub for version control allowed for efficient collaboration, code reviews, and management of different development branches.

Test Driven Development Reflection

We have gained much knowledge from successful and unsuccessful applications of DevOps and TDD (Test-Driven Development) throughout our project. For example, TDD was quite helpful to us when we followed it correctly. It was easier and our code was better since we had a clear plan before we started writing code. However, there were instances when we deviated from TDD, such as when

creating the views for the addition of new reactions. The testing was not properly set up; therefore, issues weren't discovered until we were nearly finished. It was a little difficult at first because we were all new to this, but when we did it correctly, we could see the advantages. TDD is undoubtedly worthwhile because it allowed us to build cleaner, more effective code.

Group/Project work Reflection

We have discovered from our project/group work that successful collaboration requires clear planning, efficient communication, and a willingness on the part of all team members. Efficient communication guarantees that all parties agree, and well-planned planning keeps us focused on our objectives and organized. Above all, the ability of each team member to contribute and adjust is essential to overcome obstacles and effectively accomplish our goals.

Challenges

There were certain difficulties associated with being a member of an online-only group, most of which had to do with cooperation and communication. The lack of in-person connection made task coordination and providing progress reports more difficult. Time management was also an issue when balancing project obligations with other classes, especially continuous exams. Our group prioritized constant communication via online platforms and arranged virtual meetings to discuss project tasks and deadlines to overcome these obstacles. We also used task tracking and document sharing technology for collaboration to increase productivity and responsibility.

Conclusion

This report has detailed the development and implementation of a social media platform designed to allow users to post content and interact with others. The project utilized Django for the backend and React for the frontend, providing a robust and dynamic user experience. Scrum methodology was employed to manage the project effectively, facilitating organized teamwork and iterative development.

Key features of the platform include secure user authentication using JWT, efficient database management with Django ORM, and seamless frontend-backend integration. Despite challenges such as ensuring security and managing concurrent requests, the project successfully addressed these issues through modern development tools and practices.

Overall, the project has achieved its primary goals, resulting in a functional and user-friendly social media platform. The experience gained from this project has provided valuable insights and strengthened the technical and collaborative skills of the development team. Future improvements and expansions are anticipated to enhance the platform further.

References

1. Django Documentation

- Django Software Foundation. (n.d.). Django documentation. Retrieved from <https://docs.djangoproject.com/en/stable/>

2. Django Rest Framework

- Encode. (n.d.). Django Rest Framework. Retrieved from <https://www.django-rest-framework.org/>

3. React Documentation

- Meta Platforms, Inc. (n.d.). React documentation. Retrieved from <https://reactjs.org/docs/getting-started.html>
- 1. Scrum Framework**
 - Schwaber, K., & Sutherland, J. (2017). The Scrum Guide. Retrieved from <https://scrumguides.org/scrum-guide.html>
- 2. JSON Web Tokens (JWT)**
 - JWT.io. (n.d.). Introduction to JSON Web Tokens. Retrieved from <https://jwt.io/introduction/>
- 3. Axios Documentation**
 - Axios. (n.d.). Axios documentation. Retrieved from <https://axios-http.com/docs/intro>
- 4. PostgreSQL Full-Text Search**
 - The PostgreSQL Global Development Group. (n.d.). PostgreSQL documentation. Retrieved from <https://www.postgresql.org/docs/current/textsearch.html>
- 5. Git and GitHub**
 - Chacon, S., & Straub, B. (2014). Pro Git. Apress. Retrieved from <https://git-scm.com/book/en/v2>
- 6. Agile Methodology**
 - Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., ... Thomas, D. (2001). Manifesto for Agile Software Development. Retrieved from <https://agilemanifesto.org/>
- 7. Virtual DOM in React**
 - Meta Platforms, Inc. (n.d.). React documentation - Reconciliation. Retrieved from <https://reactjs.org/docs/reconciliation.html>
- 8. Test-Driven Development (TDD)**
 - Beck, K. (2003). Test-Driven Development: By Example. Addison-Wesley Professional.

Appendix

User Stories List

These are the user stories created by us to aid our decisions throughout development of ProjectPlatform.

User Authentication and Authorization

- As a user, I want to create a new account, providing my email and password, so that I can securely access the blog platform.
- As a user, I want to log in to the platform using my credentials, ensuring a secure and authenticated session.
- As a user, I want the platform to securely log me out to protect my account information.
- As an admin, I want to manage user roles and permissions to control access to CRUD operations.

Blog Post Management

- As a user, I want to create a new blog post, adding a title and content with rich text formatting for an engaging post.
- As a user, I want to view a list of all blog posts, allowing me to discover and read content.
- As a user, I want to update my existing blog posts, ensuring that the content is always relevant.

- As a user, I want to delete a blog post that I no longer wish to have on the platform.

Commenting System

- As a user, I want to comment on a blog post, expressing my thoughts and engaging with the content.
- As a user, I want to reply to comments, creating threaded discussions for more in-depth conversations.

User Interaction

- As a user, I want to follow other users, allowing me to stay updated on their new blog posts.
- As a user, I want to see an activity feed that displays posts from users I follow, curated based on my preferences.

Tagging and Categorization

- As a user, I want to add relevant tags to my blog posts, improving discoverability.
- As a user, I want to organize my posts into categories for thematic grouping
- Additional Features (Optional)
- As a user, I want to express my reaction to blog posts and comments using like and dislike buttons.
- As a user, I want to receive notifications for new comments, followers, or mentions.
- As a user, I want robust search functionality to easily find posts or users.
- As a user, I want the ability to save drafts of my blog posts before publishing.
- As a user, I want the platform to support multiple languages for a wider audience.

System Architecture

- As a developer, I want to choose and set up front-end technologies (React, Vue.js, or Angular) for a dynamic and interactive user interface.
- As a developer, I want to utilize Bootstrap or Tailwind CSS for styling and layout to expedite development.
- As a developer, I want to use Django as the back-end framework for rapid development and clean design.
- As a developer, I want to set up and configure the database, using SQLite for development and considering PostgreSQL for production.

Data Storage

- As a developer, I want to define Django models for users, posts, comments, tags, and categories to structure the relational database.
- As a developer, I want to implement media storage considerations for serving images and other media files related to blog posts.

Installation and Setup

- As a developer, I want clear instructions for setting up Python virtual environments and installing Django.
- As a developer, I want detailed steps for installing project dependencies using pip.
- As a developer, I want a guide for initial database setup and migrations using Django's built-in tools.

- As a developer, I want steps to launch the development server and access the application locally.

API Documentation

- As a developer, I want a detailed overview of API endpoints for authentication, blog posts, comments, and user interactions.
- As a developer, I want clear documentation on request/response formats, including JSON payloads and error handling.

UI Design

- As a designer, I want wireframes that showcase the layout and flow of the application.
- As a designer, I want guidelines for a design system, including color schemes, typography, and UI components.

Collaboration Guidelines

- As a developer, I want version control guidelines, using Git and GitHub for source code management, branching, and pull requests.
- As a developer, I want a code review process to maintain quality and consistency in contributions.
- As a team member, I want communication channels (Discord, Teams meetings) to ensure effective collaboration.

MoSCow Analysis

This helped us choose which functionalities to prioritize, as well as setting effective goals.

Must Have

User Authentication and Authorization

- User account creation
- Secure login/logout
- Admin role management

Blog Post Management

- Create, read, update, and delete (CRUD) operations for blog posts
- Rich text editing for enhanced content formatting

Commenting System

- Ability to comment on blog posts
- Threaded comments for in-depth discussions

User Interaction

- Follow mechanism for user updates
- Activity feed for posts from followed users

Tagging and Categorization

- Ability to add tags to posts
- Organizing posts into categories

Should-Have:

Additional Features (Optional)

- Reactions (like and dislike buttons)
- Notifications for new comments, followers, or mentions
- Robust search functionality

Could-Have:

Additional Features (Optional)

- Direct Messaging
- Ability to save drafts of blog posts
- Multi-language support (i18n and l10n)

Backlog

This is the TODO list feature from GitHub, we decided to use this as our backlog. It is simple, structured, and allows us to change the status of different assignments from an idea to finished.

	Title	...	Status	≡ ↑ ...
1	🕒 Utilize local or session storage for user profiles #16		Idea	▼
2	🕒 Search bar #13		Todo	▼
3	🕒 Messaging #6		Todo	▼
4	🕒 Enhance personal profile page #19		In Progress	▼
5	🕒 Reactions (likes, etc...) #12		In Progress	▼
6	🕒 Django raises exception when using only one <code>UserSerializer</code> #9		In Progress	▼
7	✅ Front-End #2		Done	▼
8	✅ Create user login #1		Done	▼
9	✅ Profiles #3		Done	▼
10	✅ Posts #4		Done	▼
11	✅ Encryption #5		Done	▼
12	✅ Signup #7		Done	▼
13	✅ Uploading images #8		Done	▼
14	✅ Followers #10		Done	▼
15	✅ Comments #11		Done	▼
16	✅ Control the use of <code>useEffect</code> in React #15		Done	▼
17	✅ Public profile pages #17		Done	▼
18	✅ Posts on public profile pages #18		Done	▼
19	✅ Reworking paths to posts and comments #20		Done	▼
20	✅ Enviroment variables #21		Done	▼

Timesheet and Contributions

Team Member	Hours Worked	Description of Work
August Karlsen	67 hours	Frontend development, UI/UX design, integration, Report
Markus Wilhelmsen	89 hours	Frontend development, state management, API integration and development, backend development
Mahmoud Allorqsosi	70 hours	Backend development, API design, database management
Tor Opdahl	72 hours	Frontend development, Backend Development, API development.

Meeting Minutes

Meeting Notes: January 31st

- Introduction
- Scrum
- Decide on product
- Familiarize with Django

Meeting Notes: February 7th

Product Vision:

- "For [target customer], who need [need or opportunity]. The product name is . . . , that enables [key benefit, reason to buy/use]. Unlike [competitor], our product provides [differentiation]."

User Stories, Personas:

- "As [a type of persona], I want [a feature] so that [benefit]."

Meeting Notes: February 14th

- MoSCow
- Report template
- Decided for product

Meeting Notes: February 21st

- Must-Have Features:
 - Database, login, users, posts, administrative user.
- Blog Direction:
 - Student platform, book sales, faculties, events, groups, direct chat.
 - Tools and Resources:
 - Utilize Planning Poker for estimation.
 - Collaborate on a shared document using Google Docs.

Meeting Notes: February 28th

- Progress Summary
- Backlog planning

Meeting Notes: March 6th

- Cancelled, TA unavailable

Meeting Notes: March 13th

- Cancelled, TA unavailable

Meeting Notes: March 20st

- Progress summary
- Backlog planning

Meeting Notes: March 27th

- Cancelled, easter break.

Meeting Notes: April 3rd

- Progress summary
- Backlog planning

Meeting Notes: April 10th

- Progress summary
- Backlog planning

Meeting Notes: April 17th

- Progress summary
- Backlog planning

Meeting Notes: April 24th

- Progress summary
- Backlog planning

Meeting Notes: May 1st

- Cancelled, holiday.

Meeting Notes: May 8th

- Report planning
- Gathering of documentation
- Formatting Template

Meeting Notes: May 15th

- Feedback on report so far

